

Institute of Engineering & Management, Kolkata

Department of Computer Science and Engineering

E-Commerce Purchase Flow Automation Framework

Selenium WebDriver + Python + pytest (Page Object Model)

Capstone Project Report

Selenium / Web UI Testing Coursework

Submitted by:

Srijani Bishnu

Mentor:

Prof. Sourav Malakar

September 29, 2026

Contents

Contents.....	2
1. Introduction	3
1.1 Objective	3
1.2 Scope.....	3
1.3 Tools and Technologies Used	3
2. System Design and Architecture	3
2.1 Layered Architecture	4
2.2 Project Folder Structure	4
3. Implementation Details	5
3.1 Configuration Layer.....	5
3.2 Base Page - Reliability Engineering	5
3.3 Page Objects - Auth, Products, Cart.....	5
3.4 Data-Driven Testing (JSON / Excel)	6
3.5 End-to-End Test Scenario	6
3.6 Execution Reporting.....	6
4. Testing and Results	6
4.1 Test Execution	6
4.2 Unit Test Verification	7
4.3 End-to-End Scenario Checklist.....	7
4.4 Screenshots	8
4.4.1 Unit Test Console Output	8
4.4.2 Registration / Account Information Page.....	8
4.4.3 Successful Signin	9
4.4.4 Cart Verification	9
4.4.5 HTML Execution Report.....	10
5. Conclusion.....	10
5.1 Summary	10
5.2 Future Enhancements	11
5.3 Repository	11

1. Introduction

1.1 Objective

The objective of this project is to design and implement a complete Web UI Test Automation Framework using Python and Selenium WebDriver. The framework automates a full customer purchase journey on a live e-commerce demo site and demonstrates industry-standard practices, including:

- Page Object Model (POM) design for maintainable, reusable page interactions
- Reliable handling of real-world UI obstacles - ad overlays, popups and native browser alerts
- Data-driven testing from JSON or Excel, with upfront validation of test data
- A self-healing test-user strategy that removes flaky, credential-dependent logins
- Screenshot capture at every step and on failure
- A self-contained HTML execution report with embedded screenshots
- Cross-browser execution (Chrome, Firefox, Edge) and headless / slow-motion modes

1.2 Scope

The framework targets the public demo application hosted at <https://automationexercise.com> as the system under test, covering the complete purchase lifecycle: user registration, login, product search, adding a product to the cart with a chosen quantity, and cart verification. The test account used for each run is created fresh and deleted again at the end of the run, so the suite has no dependency on any pre-existing account and can be re-run indefinitely without manual data reset.

1.3 Tools and Technologies Used

Tool / Library	Purpose
Python 3.9+	Core programming language
Selenium 4	Browser automation (driver managed automatically by Selenium Manager)
pytest	Test runner, fixtures and command-line configuration
openpyxl	Reading and validating Excel test data
Page Object Model	Design pattern separating locators/actions from test logic
Custom HTML report generator	Self-contained execution report with embedded screenshots

2. System Design and Architecture

2.1 Layered Architecture

The framework follows a layered design so that responsibilities are cleanly separated and the codebase stays maintainable and reusable:

- Test Layer - pytest test functions describing the purchase-flow scenario in plain, readable steps.
- Step Runner Layer - times each step, captures a screenshot, and records PASS / FAIL / WARN to the report.
- Page Object Layer - one class per page (HomePage, AuthPage, ProductsPage, CartPage) exposing actions such as login() or add_to_cart().
- Base Page Layer - shared, hardened Selenium primitives: waits, retrying clicks, ad-overlay removal, alert handling, dropdown selection.

```
Test Layer (tests/test_purchase_flow.py)
  |
  v
Step Runner (utils/steps.py) --- timing, screenshots, PASS/FAIL/WARN
  |
  v
Page Object Layer (HomePage / AuthPage / ProductsPage / CartPage)
  |
  v
Base Page (waits, retrying clicks, ad removal, alerts, dropdowns)
  |
  v
Selenium WebDriver --> Browser --> automationexercise.com
```

2.2 Project Folder Structure

```
ecommerce_v2/
├── conftest.py           # fixtures + report hooks
├── pytest.ini
├── requirements.txt
├── config/
│   └── settings.py      # URL, browser, headless, timeouts
├── data/
│   ├── test_data.json
│   └── test_data.xlsx
├── pages/
│   ├── base_page.py     # waits, retrying clicks, ad removal
│   ├── home_page.py
│   ├── auth_page.py     # login and signup
│   ├── products_page.py
│   └── cart_page.py
├── utils/
│   ├── driver_factory.py # Chrome / Firefox / Edge setup
│   ├── data_reader.py   # JSON and Excel + validation
│   ├── user_factory.py  # unique test-user generation
│   ├── steps.py         # timing, screenshots, logging
│   ├── report.py        # HTML report generator
│   └── helpers.py
├── tests/
│   ├── test_purchase_flow.py # the end-to-end scenario
│   └── unit/
```

```
└─ test_utils.py    # fast offline unit tests
```

3. Implementation Details

3.1 Configuration Layer

All environment-specific values (target URL, browser, headless mode, timeouts, slow-motion delay) are centralised in `config/settings.py` and can be overridden with environment variables, so the same code runs differently in different environments purely through configuration - no code changes required.

```
BASE_URL = os.getenv("BASE_URL", "https://automationexercise.com")
BROWSER  = os.getenv("BROWSER", "chrome").lower() # chrome|firefox|edge
HEADLESS = os.getenv("HEADLESS", "false") == "true"
DATA_FILE = os.getenv("DATA_FILE", "test_data.json") # or test_data.xlsx

EXPLICIT_WAIT = int(os.getenv("EXPLICIT_WAIT", "20"))
SLOW_MO       = float(os.getenv("SLOW_MO", "0"))      # pause before each action
STEP_PAUSE    = float(os.getenv("STEP_PAUSE", "0"))   # pause after each step
```

3.2 Base Page - Reliability Engineering

The demo application under test displays full-page Google ad overlays that intercept clicks on the Login and Signup buttons - the single biggest source of flakiness in a naive Selenium script against this site. `BasePage` addresses this directly: ad hosts are blocked at network level for Chrome and Edge, ad containers are stripped from the DOM before every interaction, and a click that is still intercepted is retried and, as a last resort, issued via JavaScript.

```
def click(self, locator, timeout=None):
    for _ in range(3):
        try:
            element = self._wait(timeout).until(
                EC.element_to_be_clickable(locator))
            self._slow(element)
            element.click()
            return
        except (ElementClickInterceptedException,
                ElementNotInteractableException,
                StaleElementReferenceException):
            self.remove_ads()
            self.scroll_to(locator)
        except UnexpectedAlertPresentException:
            self.accept_alert()
    # last resort if the overlay never clears
    self.driver.execute_script(
        "arguments[0].click();", self.presence(locator))
```

3.3 Page Objects - Auth, Products, Cart

Each page exposes intent-revealing methods rather than raw Selenium calls. `AuthPage.login()` waits for either a logged-in state or the site's own error message and raises a descriptive `AuthError` instead of leaving the test to time out with no explanation:

```
def login(self, email, password):
```

```

self.type_text(self.LOGIN_EMAIL, email)
self.type_text(self.LOGIN_PASSWORD, password)
self.click(self.LOGIN_BUTTON)

status, message = WebDriverWait(self.driver, 15).until(outcome,
    'Neither a logged-in state nor a login error appeared')
if status == 'error':
    raise AuthError(f"Login rejected for '{email}': {message}")

```

3.4 Data-Driven Testing (JSON / Excel)

Test data can be supplied as either JSON or Excel - both are parsed into an identical dictionary structure and validated before any browser is launched, so a misconfigured run fails fast with a readable list of problems rather than an obscure Selenium error midway through the test:

```

def validate_test_data(data):
    problems = []
    if data.get('user_mode') not in ('new', 'existing'):
        problems.append("user_mode must be 'new' or 'existing'")
    if not str(data.get('search', {})).get('product_keyword', '').strip():
        problems.append('search.product_keyword is empty')
    if problems:
        raise ValueError('Invalid test data:\n - ' + '\n - '.join(problems))

```

3.5 End-to-End Test Scenario

The scenario is written as a single pytest function, with each logical step wrapped in a StepRunner context manager that times the step, takes a screenshot, and logs the outcome to the execution report - whether the step passes, fails, or (for the best-effort cleanup step) only warns:

```

with steps.step('Search product') as s:
    home.go_to_products()
    products.search(keyword)
    found = products.result_count()
    assert found > 0, f"No products found for '{keyword}'"
    s.details = f"Search '{keyword}' returned {found} product(s)"

```

3.6 Execution Reporting

utils/report.py builds a single, self-contained HTML file after every run: an overall PASS/FAIL banner, one row per step with its status, duration and details, and every screenshot embedded directly in the file as base64 data - so the report can be emailed or archived as a single artifact with no separate screenshots folder required.

4. Testing and Results

4.1 Test Execution

The full suite (unit tests and the end-to-end browser scenario) is executed with:

```

pip install -r requirements.txt

```

```
pytest
```

The end-to-end browser scenario can also be run on its own, and adjusted through environment variables:

```
pytest tests/test_purchase_flow.py
HEADLESS=true pytest tests/test_purchase_flow.py
BROWSER=firefox pytest tests/test_purchase_flow.py
SLOW_MO=1 STEP_PAUSE=2 pytest tests/test_purchase_flow.py
```

4.2 Unit Test Verification

Before relying on the browser scenario, the offline logic - data loading, validation, unique-user generation, and the report generator - was verified with a dedicated unit test suite that needs no browser and runs in well under a second:

Test	Area covered	Result
test_parse_money	Utility	Passed
test_safe_filename_and_short_error	Utility	Passed
test_new_user_email_is_unique_and_profile_untouched	User factory	Passed
test_json_and_excel_data_are_identical	Data reader	Passed
test_bundled_data_is_valid	Data reader	Passed
test_validation_reports_every_problem	Validation	Passed
test_validation_rejects_unknown_mode	Validation	Passed
test_excel_types_are_coerced	Data reader	Passed
test_report_is_self_contained_html	Report generator	Passed

Result: 9 passed, 0 failed.

4.3 End-to-End Scenario Checklist

The following steps make up the browser-based purchase flow. Run the suite locally against the live application (Section 4.1) and record the actual result and duration of each step from your own execution report:

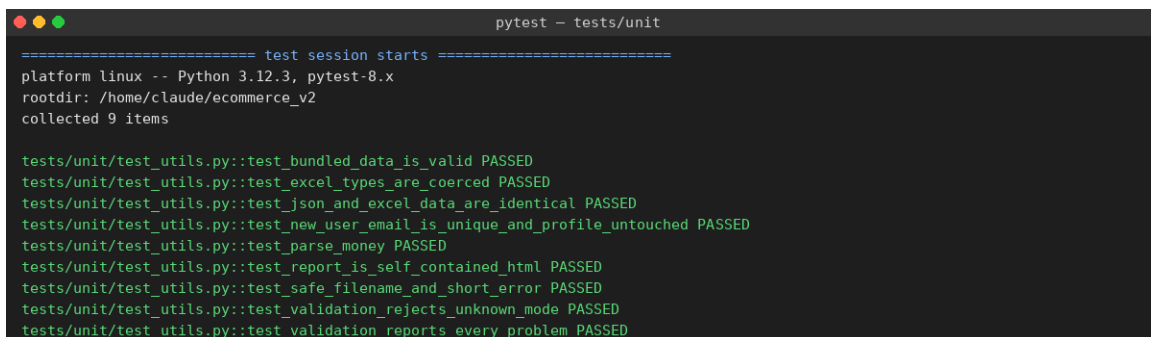
Step	Type	Expected outcome
Launch browser and open application	Positive (setup)	Home page loads, popups dismissed
Register a new user	Positive	'ACCOUNT CREATED!' is displayed
Logout	Positive	Redirected to the login page

Step	Type	Expected outcome
Login with the new credentials	Positive	'Logged in as <name>' is displayed
Search for a product	Positive	At least one search result is returned
Update quantity and add to cart	Positive	'Added!' popup is displayed
Verify cart details	Positive (assertion-heavy)	Name, unit price, quantity and total all match
Cleanup - delete test account	Best effort	Account removed; never fails the test on its own

4.4 Screenshots

The screenshot below is genuine output captured while verifying this framework's offline logic. The remaining figures are placeholders for you to replace with your own screenshots after running the end-to-end suite locally - the report and screenshots folder are generated fresh by every run (see Section 3.6).

4.4.1 Unit Test Console Output



```

pytest - tests/unit

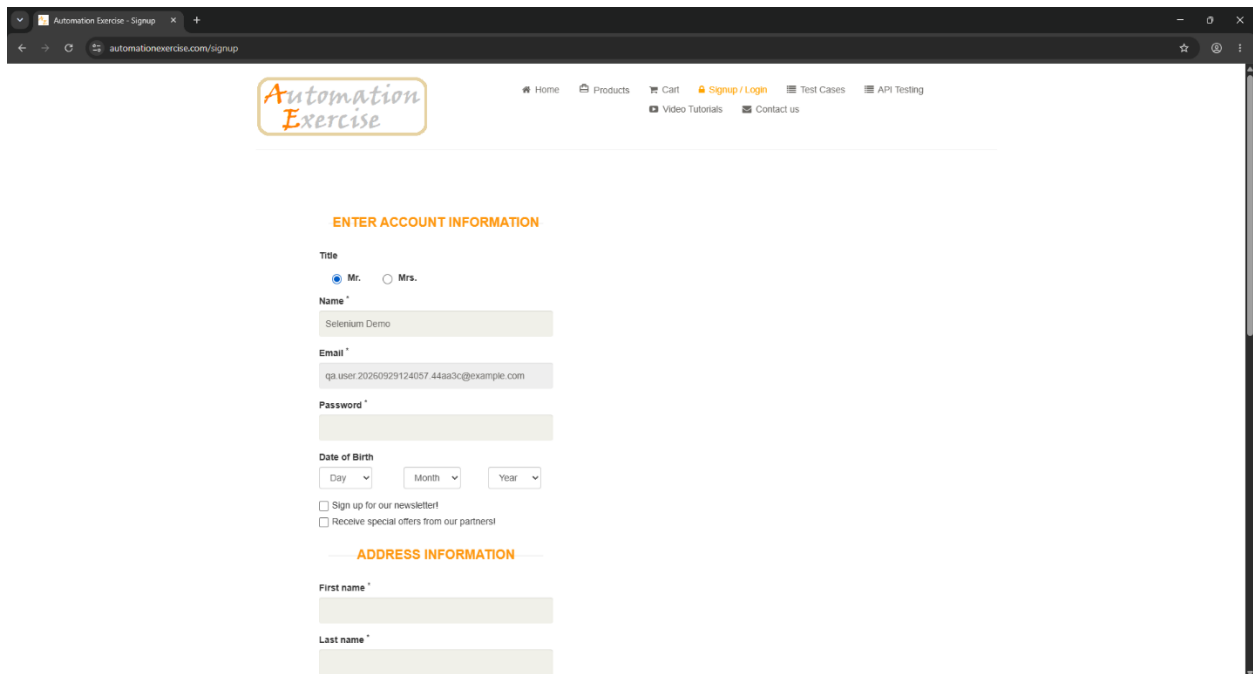
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.x
rootdir: /home/claude/ecommerce_v2
collected 9 items

tests/unit/test_utils.py::test_bundled_data_is_valid PASSED
tests/unit/test_utils.py::test_excel_types_are_coerced PASSED
tests/unit/test_utils.py::test_json_and_excel_data_are_identical PASSED
tests/unit/test_utils.py::test_new_user_email_is_unique_and_profile_untouched PASSED
tests/unit/test_utils.py::test_parse_money PASSED
tests/unit/test_utils.py::test_report_is_self_contained_html PASSED
tests/unit/test_utils.py::test_safe_filename_and_short_error PASSED
tests/unit/test_utils.py::test_validation_rejects_unknown_mode PASSED
tests/unit/test_utils.py::test_validation_reports_every_problem PASSED

```

Figure 4.1: Offline unit test suite - 9 passed, 0 failed (genuine output)

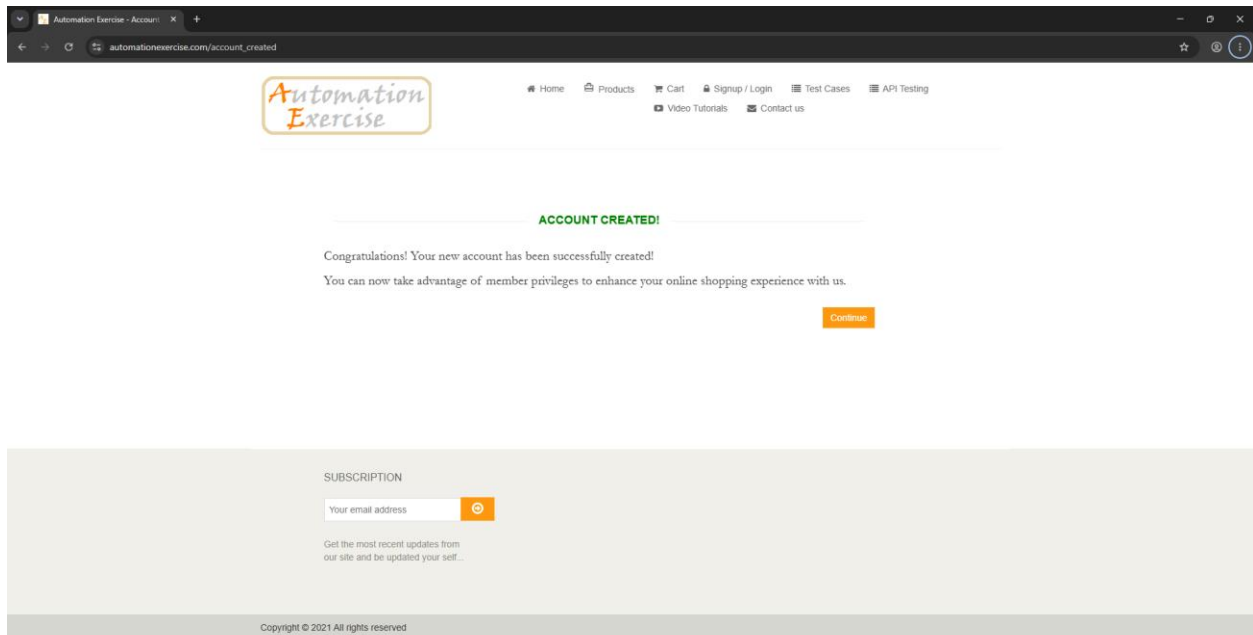
4.4.2 Registration / Account Information Page



The screenshot shows a web browser window with the URL `automationexercise.com/signup`. The page features the "Automation Exercise" logo and a navigation menu with links to Home, Products, Cart, Signup / Login, Test Cases, API Testing, Video Tutorials, and Contact us. The main content area is titled "ENTER ACCOUNT INFORMATION" and contains a form with the following fields: Title (radio buttons for Mr. and Mrs.), Name (text input with "Selenium Demo"), Email (text input with "qa.user.20260929124057.44aa3c@example.com"), Password (text input), Date of Birth (Day, Month, and Year dropdowns), and two checkboxes for newsletter and special offers. Below this is the "ADDRESS INFORMATION" section with First name and Last name text inputs.

Figure 4.2

4.4.3 Successful Signin



The screenshot shows a web browser window with the URL `automationexercise.com/account_created`. The page features the "Automation Exercise" logo and the same navigation menu as Figure 4.2. The main content area has a green "ACCOUNT CREATED!" heading, followed by a congratulatory message: "Congratulations! Your new account has been successfully created! You can now take advantage of member privileges to enhance your online shopping experience with us." Below this is an orange "Continue" button. At the bottom, there is a "SUBSCRIPTION" section with a text input for "Your email address" and a plus icon button. Below the input is the text: "Get the most recent updates from our site and be updated your self...". The footer contains the text "Copyright © 2021 All rights reserved".

Figure 4.3: Screenshot of successful sign in

4.4.4 Cart Verification

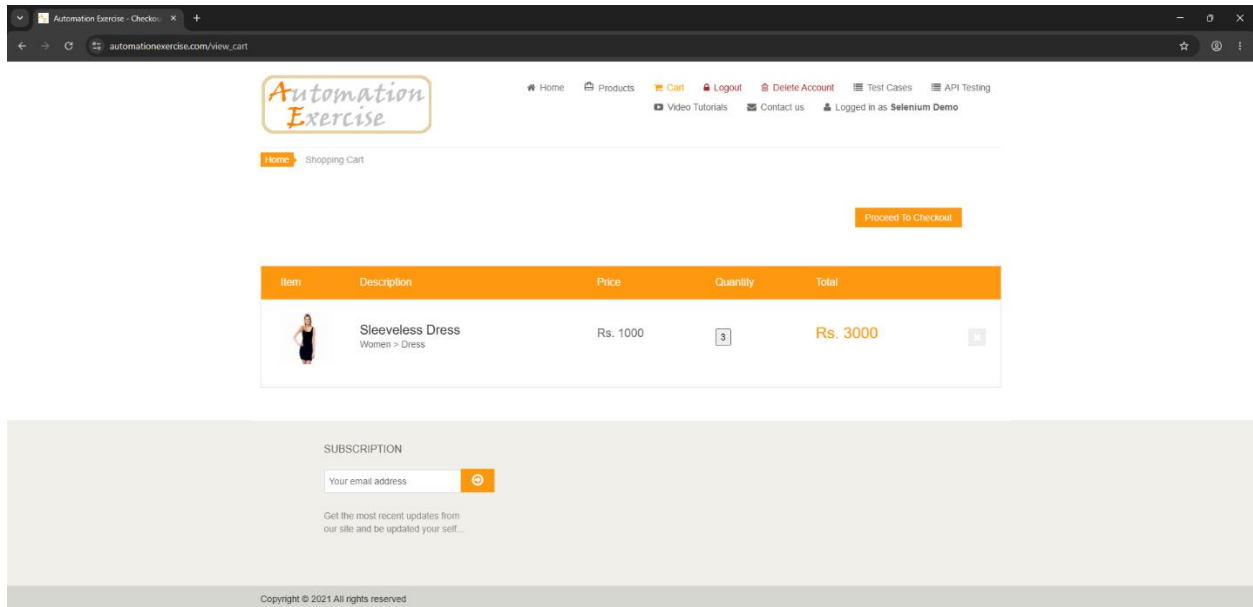


Figure 4.4: Screenshot of the cart page

4.4.5 HTML Execution Report

```
reports > <> execution_report.html > ...
1  <!DOCTYPE html><html><head><meta charset='utf-8'><title>E-Commerce Purchase Flow - Execution Report</title><style>
2  body{font-family:Segoe UI,Arial,sans-serif;margin:0;background:■ #f4f5f7;color:□ #222}
3  .wrap{max-width:1200px;margin:0 auto;padding:28px}
4  h1{margin:0 0 6px 0}h2{margin:28px 0 10px 0}
5  .banner{padding:14px 20px;border-radius:8px;color:■ #fff;font-size:20px;font-weight:600;margin:16px 0}
6  .PASSED{background:□ #2e7d32}.FAILED{background:■ #c62828}
7  .cards{display:flex;gap:14px;flex-wrap:wrap;margin-bottom:8px}
8  .card{background:■ #fff;border-radius:8px;padding:12px 20px;box-shadow:0 1px 3px □ #0002;min-width:110px}
9  .card b{display:block;font-size:24px}
10 table{border-collapse:collapse;width:100%;background:■ #fff;box-shadow:0 1px 3px □ #0002}
11 th,td{padding:10px 12px;border-bottom:1px solid ■ #e5e5e5;text-align:left;vertical-align:top;font-size:14px}
12 th{background:□ #263238;color:■ #fff}
13 .badge{padding:2px 10px;border-radius:12px;color:■ #fff;font-weight:600;font-size:12px}
14 .PASS{background:□ #2e7d32}.FAIL{background:■ #c62828}.WARN{background:■ #ef6c00}
15 .passed{background:□ #2e7d32}.failed{background:■ #c62828}.skipped{background:■ #757575}
16 details summary{cursor:pointer;color:■ #1565c0}
17 details img{max-width:520px;border:1px solid ■ #ccc;margin-top:8px}
18 .meta td:first-child{font-weight:600;width:200px}
19 </style></head><body><div class='wrap'><h1>E-Commerce Purchase Flow - Execution Report</h1><div class='banner PASSED'>Overa
```

Figure 4.5: Screenshot of reports/execution_report.html

5. Conclusion

5.1 Summary

This project implements a reliable, maintainable Selenium WebDriver automation framework for a complete e-commerce purchase flow, built on the Page Object Model. It specifically addresses the two failure modes that most commonly break UI automation against real-world sites - ad overlays intercepting clicks, and stale or duplicated test credentials - by removing ad overlays before every

interaction and generating a fresh, unique test user on every run. The framework is data-driven from JSON or Excel, validates its own inputs before launching a browser, and produces a single self-contained HTML report with every step timed, screenshotted and logged.

5.2 Future Enhancements

- Integrate with a CI/CD pipeline (GitHub Actions) for automated runs on every push
- Add parallel execution (pytest-xdist) to reduce overall run time across browsers
- Extend coverage to checkout, payment, and order-history flows
- Add visual regression checks on key pages using screenshot comparison
- Publish results to a richer reporting tool (e.g. Allure) alongside the existing HTML report

5.3 Repository

The complete source code for this project is available at:

<https://github.com/srijan-i/ecommerce-selenium-automation>